

Project Explain

Employee Background Check and Resume Analysis System

A local-first AI workflow for HR document screening, verification, and reporting.

Project Overview

This project is a local-first AI-assisted employee background check and resume analysis system. It accepts uploaded documents, extracts text with OCR, verifies key fields, performs retrieval against a local corpus, runs external verification connectors, assigns a risk score, generates a report, stores the case in SQLite, and shows the result in a dashboard for HR review.

Architecture

The logical architecture is: User or HR Portal -> Document Upload Layer -> OCR and Text Processing -> Data Normalization -> Verification Engine -> Fraud Detection and Risk Scoring -> Human Review if needed -> Report Generation -> HR Dashboard and System Storage.

End-to-End Flow

1. The user uploads a text file, image, or PDF through the demo page or API.
2. The file is saved locally in data/uploads.
3. OCR or text extraction produces raw_text, fields, and confidence.
4. Field verification checks required values such as name, date, and ID number.
5. The retrieval layer searches the local corpus for related text.
6. External verification connectors run in mock mode by default or live mode when configured.
7. Fraud signals and risk score are calculated.
8. A report is produced using template mode or Ollama mode.
9. The case is stored in SQLite.
10. The dashboard displays totals, review queue items, charts, and recent cases.

Files and Responsibilities

src/app.py is the orchestration layer. It exposes the FastAPI routes, initializes the database on startup, stores uploads, and manages the system state.
src/ocr.py handles extraction and analysis. It supports text, image, and PDF input, extracts fields, builds verification queries, and returns structured data.
src/rag.py provides local retrieval. It loads corpus text, builds an in-memory embedding index, and returns similar documents.
src/connectors.py implements the external verification skeleton. It includes mock employment and identity checks.
src/llm_report.py manages optional LLM report generation. It builds the Ollama prompt, calls the local Ollama API, and formats the response.
src/db.py owns storage and analytics. It creates the SQLite schema, saves cases, reads cases, lists pending reviews, and manages reports.
src/insights.py parses resume text for dashboard charts. It extracts experience buckets, education buckets, and skill sets.
src/templates/upload.html is the demo analyzer page. It has the upload form, report-only form, and output section.
src/templates/dashboard.html is the HR dashboard page. It shows KPI cards, a review overview, recent cases, and aggregate insights.
docs/DESIGN.md explains the intended logical architecture, the local-first model policy, and why Ollama and Minimax were chosen.
docs/MODEL_SELECTION_REPORT.md explains the model choice tradeoffs and why sentence-transformers/Embedder was selected.

Ollama

src/llm_report.py is the Ollama integration point.

If REPORT_GENERATION_MODE is not set to ollama, the system uses the template report immediately.

If REPORT_GENERATION_MODE=ollama, the code sends a POST request to /api/generate on the local Ollama API.

If Ollama returns an empty response, fails, or is unavailable, the system safely falls back to the template report immediately.

This keeps the project local-first and allows demo continuity even when the model server is down.

Dashboard Behavior

src/templates/dashboard.html renders the HR dashboard UI. It fetches /dashboard/summary to populate the KPI cards, recent cases, and aggregate insight data. After a fresh upload, it

also replaces the charts with the latest upload's resume_insights so the user immediately sees that resume's experience, education, and top skills.

Demo Story

A strong demo sequence is: open the dashboard, upload a resume, show the extracted assessment and charts. The project demonstrates an end-to-end document analysis workflow with OCR, retrieval, verification, risk scoring